

Course Project: AI-powered Vulnerability & Exposure Support Agent

1. Project Goal

Build an **AI-powered Vulnerability & Exposure Support Agent** that helps security teams and developers answer questions about organizational security hardening policies and retrieve asset-specific vulnerability and exposure data by combining:

1. **RAG (Retrieval-Augmented Generation)** — semantic search over security policies.
2. **LLM with tool use** — The LLM decides when to retrieve policies and when to query the database.
3. **MCP tools** — A Model Context Protocol server exposing read/write access to the SQLite database.
4. **Two interfaces** — Command-line chat and browser-based chat with live “thinking” steps.
5. **Agentic loop** — The agent receives a goal and autonomously decides the next step (use tools or respond).
6. **Long conversations** — The agent maintains memory of the current conversation.

The agent should handle questions like “What is the hardening policy for Linux servers?”, “What vulnerabilities were found on asset server-prod-01?”, and “How do I remediate CVE-2025-XXXX?” by retrieving the right policy text and querying the right database tables.

The agent should also be able to **perform operations** on behalf of the user. In this case it will be able to open a remediation ticket for the user by adding it to the remediation_tickets table.

What you receive (starter materials):

- Policy knowledge base (dataset_design/01_knowledge_base_documents.md) - 21 policy documents
- Database schema design (dataset_design/02_sql_schema_definition.md)
- Sample data specification (dataset_design/03_generated_dataset_samples.md)
- Example support scenarios for testing (dataset_design/04_example_support_scenarios.md)
- Database initialization script (database/init_db.py) - creates database/security.db with sample data

What you must build: All application code — RAG indexing & retrieval, MCP server, agent logic, CLI, and web UI.

2. High-level Architecture



3. General Requirement:

1. The core is an agentic loop using LLM, tools, and memory.
2. The agent must support multiple users simultaneously. All LLM and API calls should be asynchronous. Use `async` and `await` where required.
3. LLM — Use Anthropic Haiku 4.5 or a model with similar capabilities.
4. Memory — Keep conversation history (up to 20 messages).
5. MCP — Database access is done exclusively through the MCP server.
6. RAG — The knowledge base (policies) will be kept in a RAG (Chroma DB) and exposed as a tool for the agent.
7. The agent will have both CLI and Web interfaces.
8. Web UI update with websocket to keep smooth web experience.
9. The agent should help the user to create a ticket if needed (insert a row in the `remediation_tickets` table).
10. **Guardrails** — the agent authenticates the user and should not allow to get unsafe information like other users' information and table information.

4. Knowledge Base Requirements:

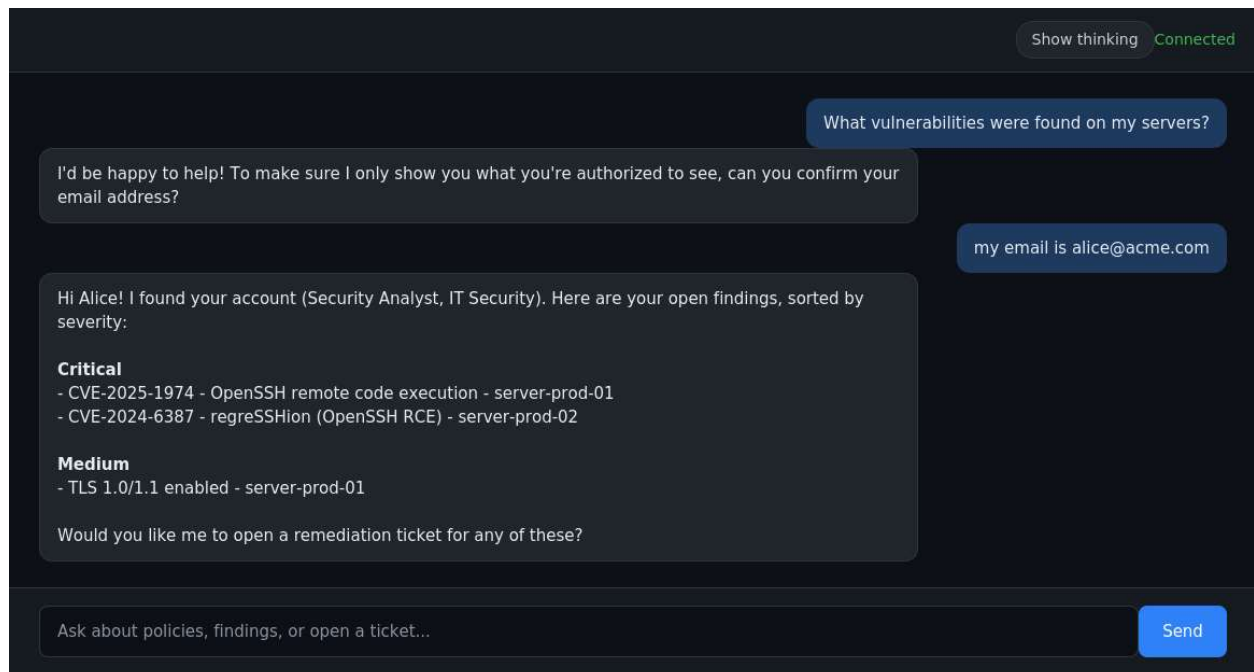
1. Contains security hardening policies per server type & version, remediation guides, OWASP Top 10, CIS Benchmarks, and secure coding standards.
2. Use Chroma DB locally with sentence-transformers/all-MiniLM-L6-v2 model.
3. No chunking is required — we want to give the agent the full policy document.
4. Return top 5 most relevant documents per query (configurable).

5. Database Requirements:

1. Use SQLite on local server.
2. Use init_db.py to create the tables and content.
3. The DB should expose read and write operation. The write operations should be used to create a ticket in the remediation_tickets table.
4. You can use SQLite Viewer Extension on Cursor to view structure and content.

6. Web UI Requirements:

1. The Web UI should be a simple chat — the user enters a prompt, and the agent answers.
2. There should also be an option to see the thinking process for debug purposes. The thinking process are the intermediate steps of the agent before inserting the final results. Including reasoning and tool use.
3. See example of UI below.



7. Deployment:

1. Should be able to deploy on an AWS EC2 server.
2. Deploy on an EC2 server and make sure you can access from your own PC.
3. Remember: Shutdown the server when not being used. To avoid excessive payments.

8. Stretch Goals:

1. Wrap the agent in a docker container.
2. Separate the web client from the agent server, so the web will be the client and the agent will be the server. You still need to use WebSocket communication between the agent and the client.
3. Have an option for fallback model using LiteLLM. For example, use Haiku 4.5 as main model, but if the request to Anthropic is not successful, fallback to Gemini 3.5 Flash.

Implementation Plan:

- **Phase 1:** Build the RAG solution for the knowledge base of security policies. The agent should know how to respond to questions about the organizational security policies. CLI interface.
- **Phase 2:** The agent should be able to answer questions from the database through the MCP server. The agent should be able to authenticate the user through their email and make sure to answer questions only for the relevant user — the assets and findings they own.
- **Phase 3:** Build the Web UI interface. Support multiple sessions (users) at the same time.
- **Phase 4:** Deploy the agent on an AWS EC2 server.

9. Delivery

Delivery of the final project should include a zip file with:

1. Code files and related files.
2. Requirement.txt file
3. README.md which explains how to run the project.
4. A short video that shows a example of Q&A session.

Send by mail to eransi@gmail.com

Course Project: AI Security Support Agent Database Schema (SQLite)
Unified schema: users, assets, findings (typed), scans, scan_results, risk_acceptances, patch_history, remediation_tickets - database/security.db

